Crime Management System - Complete File Documentation

Table of Contents

1. Project Root Files
2. Java Source Files
   - Main Application
   - Entity Classes
   - DAO Classes
   - Service Classes
   - UI Classes
   - Utility Classes
3. Configuration Files
4. Database Files
5. Script Files
6. Documentation Files
7. Test Files

Project Root Files

`pom.xml`

Purpose: Maven Project Object Model file that defines project configuration, dependencies, and build settings.

Key Components: - Defines project metadata (groupId, artifactId, version) - Manages Spring Boot dependencies (version 3.2.0) - Includes MySQL connector, HikariCP connection pool, FlatLaf UI library, BCrypt for password hashing - Configures Spring Boot Maven plugin with main class `com.crimemanagement.DesktopLauncher` - Sets Java version to 17

Usage: Used by Maven to build the project, manage dependencies, and package the application into a JAR file.

Java Source Files

Main Application

`DesktopLauncher.java` **Location**: `src/main/java/com/crimemanagement/DesktopLauncher.java`

Purpose: Main entry point of the desktop application. Initializes the Swing UI and launches the login dialog.

Key Functionality: - Sets up FlatLaf Look and Feel for modern UI appearance - Creates and displays the login dialog - If login is successful, creates and displays the main window with the user's role and username - Exits the application if login fails

Usage: Run command: `mvn spring-boot:run` or `java -jar target/crime-management-system-1.0.0.jar`

Entity Classes

`Criminal.java` **Location**: `src/main/java/com/crimemanagement/entity/Criminal.java`

Purpose: JPA entity representing a criminal record in the database.

Key Fields: - `criminalId`: Primary key (auto-generated) - `firstName`, `lastName`, `alias`: Personal identification - `dateOfBirth`: Date of birth - `address`: Criminal's address - `criminalRecord`: Details of past criminal activities - `createdAt`, `updatedAt`: Timestamps for record tracking

Usage: Used by `CriminalDAO` and `CriminalService` to manage criminal records in the database.

`Officer.java` **Location**: `src/main/java/com/crimemanagement/entity/Officer.java`

Purpose: JPA entity representing a police officer in the system.

Key Fields: - `officerId`: Primary key (auto-generated) - `badgeNumber`: Unique badge identifier - `firstName`, `lastName`: Officer's name - `rank`: Officer's rank (e.g., Detective, Sergeant) - `department`: Department assignment - `phoneNumber`, `email`: Contact information

Usage: Used by `OfficerDAO` and `OfficerService` to manage officer records. Linked to FIRs and cases through `assignedOfficerId`.

`FIR.java` **Location**: `src/main/java/com/crimemanagement/entity/FIR.java`

Purpose: JPA entity representing a First Information Report (FIR).

Key Fields: - `firId`: Primary key (auto-generated) - `firNumber`: Unique FIR identifier - `crimeType`: Type of crime reported - `crimeDescription`: Detailed description of the crime - `location`: Location where crime occurred - `dateOfIncident`, `dateReported`: Important timestamps - `complainantName`, `complainantPhone`, `complainantAddress`: Complainant details - `status`: Current status (Open, Closed, Under Investigation) - `assignedOfficerId`: Foreign key to officers table

Usage: Used by `FIRDAO` and `FIRService` to manage FIR records. Linked to cases through `firId`.

`Case.java` **Location**: `src/main/java/com/crimemanagement/entity/Case.java`

Purpose: JPA entity representing a criminal case in the system.

Key Fields: - `caseId`: Primary key (auto-generated) - `caseNumber`: Unique case identifier - `firId`: Foreign key linking to the related FIR - `caseTitle`: Brief title of the case - `caseDescription`: Detailed case description - `priority`: Priority level (Low, Medium, High) - `status`: Case status (Open, Closed, Under Investigation) - `assignedOfficerId`: Foreign key to officers table - `courtNumber`, `courtDate`: Court-related information

Usage: Used by `CaseDAO` and `InvestigationService` to manage case records. Linked to suspects, evidence, and case updates.

`Suspect.java` **Location**: `src/main/java/com/crimemanagement/entity/Suspect.java`

Purpose: JPA entity representing a suspect linked to a case.

Key Fields: - `suspectId`: Primary key (auto-generated) - `caseId`: Foreign key linking to the case - `criminalId`: Foreign key linking to the criminal record - `status`: Suspect status (Under Investigation, Charged, Acquitted, etc.) - `notes`: Additional notes about the suspect in this case

Usage: Used by `SuspectDAO` to link criminals to specific cases, enabling tracking of which criminals are suspects in which cases.

`Evidence.java` **Location**: `src/main/java/com/crimemanagement/entity/Evidence.java`

Purpose: JPA entity representing evidence collected for a case.

Key Fields: - `evidenceId`: Primary key (auto-generated) - `caseId`: Foreign key linking to the case - `evidenceType`: Type of evidence (Document, Weapon, Digital, etc.) - `description`: Detailed description of the evidence - `collectedDate`: Date evidence was collected - `collectedBy`: Officer ID who collected the evidence - `location`: Storage location of the evidence

Usage: Used by `EvidenceDAO` to track physical and digital evidence associated with cases.

`CaseUpdate.java` **Location**: `src/main/java/com/crimemanagement/entity/CaseUpdate.java`

Purpose: JPA entity representing updates or progress notes for a case.

Key Fields: - `updateId`: Primary key (auto-generated) - `caseId`: Foreign key linking to the case - `updateType`: Type of update (Status Change, New Evidence, Court Hearing, etc.) - `description`: Detailed description of the update - `updatedBy`: Officer ID who made the update - `updateDate`: Timestamp of the update

Usage: Used by `CaseUpdateDAO` to maintain an audit trail of case progress and changes.

`UserAccount.java` **Location**: `src/main/java/com/crimemanagement/entity/UserAccount.java`

Purpose: JPA entity representing a user account for system authentication.

Key Fields: - `userId`: Primary key (auto-generated) - `username`: Unique username for login - `passwordHash`: BCrypt-hashed password - `role`: User role (ADMIN, OFFICER, COURT) - `isActive`: Boolean flag for account activation status

Usage: Used by `UserAccountDAO` and `AuthService` for user authentication and role-based access control.

DAO Classes

`BaseDAO.java` **Location**: `src/main/java/com/crimemanagement/dao/BaseDAO.java`

Purpose: Generic interface defining standard CRUD operations for all DAO implementations.

Key Methods: - `create(T entity)`: Create a new record - `findById(K id)`: Find record by primary key - `findAll()`: Retrieve all records - `update(T entity)`: Update an existing record - `deleteById(K id)`: Delete a record by ID - `existsById(K id)`: Check if record exists - `count()`: Get total count of records - `closeResources()`: Utility method for closing database resources

Usage: All DAO classes implement this interface to ensure consistent database operations across the application.

`CriminalDAO.java` **Location**: `src/main/java/com/crimemanagement/dao/CriminalDAO.java`

Purpose: Data Access Object for criminal records. Implements `BaseDAO<Criminal, Integer>`.

Key Functionality: - CRUD operations for criminal records - Search functionality by name, alias, or address - Validation and error handling for database operations

Usage: Used by `CriminalService` and `CriminalsPanel` to interact with the criminals table in the database.

`OfficerDAO.java` **Location**: `src/main/java/com/crimemanagement/dao/OfficerDAO.java`

Purpose: Data Access Object for officer records. Implements `BaseDAO<Officer, Integer>`.

Key Functionality: - CRUD operations for officer records - Search by badge number, name, department, or rank - Validation of unique badge numbers

Usage: Used by `OfficerService` and `OfficersPanel` to manage officer records. Also used to populate officer dropdowns in other panels.

`FIRDAO.java` **Location**: `src/main/java/com/crimemanagement/dao/FIRDAO.java`

Purpose: Data Access Object for FIR records. Implements `BaseDAO<FIR, Integer>`.

Key Functionality: - CRUD operations for FIR records - Search by FIR number, crime type, location, or complainant name - Filtering by status and date ranges

Usage: Used by `FIRService` and `FIRsPanel` to manage FIR records. Also used by `CaseDAO` to link cases to FIRs.

`CaseDAO.java` **Location**: `src/main/java/com/crimemanagement/dao/CaseDAO.java`

Purpose: Data Access Object for case records. Implements `BaseDAO<Case, Integer>`.

Key Functionality: - CRUD operations for case records - Search by case number, title, status, or priority - Filtering by assigned officer and date ranges - Linking cases to FIRs

Usage: Used by `InvestigationService` and `CasesPanel` to manage case records. Also used by `SuspectDAO`, `EvidenceDAO`, and `CaseUpdateDAO` to link related records.

`SuspectDAO.java` **Location**: `src/main/java/com/crimemanagement/dao/SuspectDAO.java`

Purpose: Data Access Object for suspect records linking criminals to cases.

Key Functionality: - CRUD operations for suspect records - Finding suspects by case ID or criminal ID - Managing the relationship between cases and criminals

Usage: Used by `SuspectsPanel` to manage suspect assignments. Enables tracking which criminals are suspects in which cases.

`EvidenceDAO.java` **Location**: `src/main/java/com/crimemanagement/dao/EvidenceDAO.java`

Purpose: Data Access Object for evidence records.

Key Functionality: - CRUD operations for evidence records - Finding evidence by case ID or collecting officer - Search functionality by keyword or evidence type

Usage: Used by `SearchService` to retrieve evidence information when searching cases. Maintains chain of custody for evidence.

`CaseUpdateDAO.java` **Location**: `src/main/java/com/crimemanagement/dao/CaseUpdateDAO.java`

Purpose: Data Access Object for case update/progress records.

Key Functionality: - CRUD operations for case updates - Finding updates by case ID or update type - Maintaining chronological order of case progress

Usage: Used internally to track case history and provide audit trails for case modifications.

`UserAccountDAO.java` **Location**: `src/main/java/com/crimemanagement/dao/UserAccountDAO.java`

Purpose: Data Access Object for user account records.

Key Functionality: - CRUD operations for user accounts - Finding users by username or role - Password hash management (BCrypt) - Validation of unique usernames

Usage: Used by `AuthService` for authentication and `AdminService` for user management. Critical for role-based access control.

Service Classes

`AuthService.java` **Location**: `src/main/java/com/crimemanagement/service/AuthService.java`

Purpose: Service class handling user authentication and authorization.

Key Functionality: - `authenticate(username, password)`: Validates user credentials using BCrypt - `ensureDefaultAdmin()`: Creates a default admin user if none exists - Password verification against BCrypt hashes

Usage: Used by `LoginDialog` to authenticate users and determine their roles for system access.

`AdminService.java` **Location**: `src/main/java/com/crimemanagement/service/AdminService.java`

Purpose: Service class for administrative operations, primarily user account management.

Key Functionality: - `createUser(userAccount)`: Creates new user accounts with password hashing - `updateUser(userAccount)`: Updates existing user accounts - `deleteUser(userId)`: Deletes user accounts (with validation) - `getAllUsers()`: Retrieves all user accounts - `getUsersByRole(role)`: Filters users by role - Prevents deletion of the last admin user

Usage: Used by `AdminPanel` to manage user accounts. Only accessible to ADMIN role users.

`OfficerService.java` **Location**: `src/main/java/com/crimemanagement/service/OfficerService.java`

Purpose: Service class for officer-related business logic.

Key Functionality: - CRUD operations for officers - Validation of badge numbers (must be unique) - Search and filtering capabilities - Officer assignment validation

Usage: Used by `OfficersPanel` to manage officer records and by other services to validate officer assignments.

`CriminalService.java` **Location**: `src/main/java/com/crimemanagement/service/CriminalService.java`

Purpose: Service class for criminal-related business logic.

Key Functionality: - CRUD operations for criminals - Search by name, alias, or address - Validation of required fields - Criminal record management

Usage: Used by `CriminalsPanel` to manage criminal records and by `SuspectsPanel` to link criminals to cases.

`FIRService.java` **Location**: `src/main/java/com/crimemanagement/service/FIRService.java`

Purpose: Service class for FIR-related business logic.

Key Functionality: - CRUD operations for FIRs - Search and filtering by multiple criteria - Status management (Open, Closed, Under Investigation) - Officer assignment to FIRs

Usage: Used by `FIRsPanel` to manage FIR records. FIRs are the foundation for creating cases.

`InvestigationService.java` **Location**: `src/main/java/com/crimemanagement/service/InvestigationSer`

Purpose: Service class for case management and investigation workflows.

Key Functionality: - CRUD operations for cases - Case status management - Priority assignment - Court information management - Linking cases to FIRs - `getCaseById()`: Retrieves case with all related information - `updateCase()`: Updates case details

Usage: Used by `CasesPanel` to manage case records. Cases are the central entity for investigation tracking.

`CourtService.java` **Location**: `src/main/java/com/crimemanagement/service/CourtService.java`

Purpose: Service class for court-related operations.

Key Functionality: - Case management for court users - Court date and hearing management - Read-only access to case records for COURT role

Usage: Used by `CasesPanel` when accessed by COURT role users. Provides court-specific case management functionality.

`SearchService.java` **Location**: `src/main/java/com/crimemanagement/service/SearchService.java`

Purpose: Service class for global search functionality across all entities.

Key Functionality: - `search(keyword)`: Searches across officers, criminals, FIRs, cases, and evidence - Returns aggregated search results with entity type and relevance - Keyword matching in multiple fields - Case details retrieval with related information

Usage: Used by search functionality in various panels to find records across the entire system.

`ComplaintService.java` **Location**: `src/main/java/com/crimemanagement/service/ComplaintService.jav`

Purpose: Service class for handling complaint-related operations (legacy/web functionality).

Key Functionality: - Complaint processing - Attachment management

Usage: May be used for future web interface or complaint processing features.

UI Classes

`MainWindow.java` **Location**: `src/main/java/com/crimemanagement/ui/MainWindow.java`

Purpose: Main application window that serves as the container for all panels and tabs.

Key Functionality: - Creates tabbed interface with role-based tab visibility - Manages menu bar with user dropdown and access information - Displays different tabs based on user role: - ADMIN: All tabs including Admin panel - OFFICER: Officers, Criminals, FIRs, Cases, Suspects (full access) - COURT: Criminals, FIRs, Cases, Suspects (read-only, but can add/remove cases) - Shows user information (username, role, access level) in menu bar - Handles window closing and application exit

Usage: Launched after successful login. Central hub for all application functionality.

`LoginDialog.java` **Location**: `src/main/java/com/crimemanagement/ui/LoginDialog.java`

Purpose: Login dialog for user authentication.

Key Functionality: - Displays username, password, and role selection fields - Validates user credentials using `AuthService` - Checks if selected role has users in the database - Validates that authenticated user's role matches selected role - Returns user role and username upon successful login - Role enum: ADMIN, OFFICER, COURT

Usage: First screen shown when application starts. Must authenticate before accessing main window.

`AdminPanel.java` **Location**: `src/main/java/com/crimemanagement/ui/AdminPanel.java`

Purpose: Admin-only panel for user account management.

Key Functionality: - Displays table of all user accounts - Add new users with role selection (ADMIN, OFFICER, COURT) - Edit existing user accounts (username, password, role) - Delete user accounts (with validation to prevent deleting last admin) - Search and filter users - Refresh user list

Usage: Only visible to ADMIN role users. Provides complete user management capabilities.

`OfficersPanel.java` **Location**: `src/main/java/com/crimemanagement/ui/OfficersPanel.java`

Purpose: Panel for managing police officer records.

Key Functionality: - Displays table of all officers with sorting and searching - Add new officers (badge number, name, rank, department, contact info) - Edit existing officer records - Delete officers (with validation) - Search by badge number, name, department, or rank - Role-based access: ADMIN and OFFICER can add/edit/delete, COURT is read-only

Usage: Used by ADMIN and OFFICER roles to manage the officer database. Officers are assigned to FIRs and cases.

`CriminalsPanel.java` **Location**: `src/main/java/com/crimemanagement/ui/CriminalsPanel.java`

Purpose: Panel for managing criminal records.

Key Functionality: - Displays table of all criminals with sorting and searching - Add new criminals (name, alias, DOB, address, criminal record) - Edit existing criminal records - Delete criminals - Search by name, alias, or address - Click-to-view full text for truncated "Address" and "Criminal Record" columns - Role-based access: ADMIN and OFFICER can add/edit/delete, COURT is read-only

Usage: Used to maintain the criminal database. Criminals can be linked to cases as suspects.

`FIRsPanel.java` **Location**: `src/main/java/com/crimemanagement/ui/FIRsPanel.java`

Purpose: Panel for managing First Information Reports (FIRs).

Key Functionality: - Displays table of all FIRs with sorting and searching - Add new FIRs (FIR number, crime details, location, dates, complainant info, assigned officer) - Edit existing FIR records - Delete FIRs - Search by FIR number, crime type, location, or complainant name - Click-to-view full text for truncated "Crime Description" and "Complainant Address" columns - Role-based access: ADMIN and OFFICER can add/edit/delete, COURT is read-only

Usage: Used to manage FIR records. FIRs are the foundation for creating cases in the system.

`CasesPanel.java` **Location**: `src/main/java/com/crimemanagement/ui/CasesPanel.java`

Purpose: Panel for managing criminal cases.

Key Functionality: - Displays table of all cases with sorting and searching - Add new cases (case number, linked FIR, title, description, priority, status, assigned officer, court info) - Edit existing case records - Delete cases - Search by case number, title, status, or priority - Click-to-view full text for truncated "Title" column - Role-based access: - ADMIN and COURT can add/edit/delete cases - OFFICER is read-only (cannot add or remove cases, as this is a court function)

Usage: Central panel for case management. Cases track the investigation progress and link to suspects, evidence, and updates.

`SuspectsPanel.java` **Location**: `src/main/java/com/crimemanagement/ui/SuspectsPanel.java`

Purpose: Panel for managing suspect assignments (linking criminals to cases).

Key Functionality: - Displays table of all suspect assignments with sorting and searching - Add new suspect assignments (select case and criminal from dropdowns, status, notes) - Edit existing suspect records - Delete suspect assignments - Search by case number or criminal name - Click-to-view full text for truncated criminal and case information - Role-based access: ADMIN and OFFICER can add/edit/delete, COURT is read-only

Usage: Used to link criminals to cases as suspects. Enables tracking which criminals are suspects in which cases.

`StatsPanel.java` **Location**: `src/main/java/com/crimemanagement/ui/StatsPanel.java`

Purpose: Panel for displaying system statistics and dashboard information.

Key Functionality: - Displays counts of officers, criminals, FIRs, cases, suspects, evidence - Shows statistics charts and graphs - Provides overview of system activity

Usage: Provides administrators and officers with system overview and statistics.

`UIUtils.java` **Location**: `src/main/java/com/crimemanagement/ui/UIUtils.java`

Purpose: Utility class providing common UI helper methods for Swing components.

Key Functionality: - `applyTableDefaults(JTable)`: Configures table with sorting, tooltips, and cell rendering - Custom cell renderer for truncated text with ellipsis - Tooltip support for long text in table cells - Consistent table styling across all panels

Usage: Used by all panel classes to ensure consistent UI appearance and behavior. Handles text truncation and tooltip display.

Utility Classes

`DatabaseConnection.java` **Location**: `src/main/java/com/crimemanagement/util/DatabaseConnection.ja`

Purpose: Utility class for managing database connections using HikariCP connection pool.

Key Functionality: - Loads database configuration from `database.properties` file - Initializes HikariCP connection pool with configurable settings - Provides fallback to embedded H2 database if MySQL is unavailable - Automatically loads schema and sample data into H2 fallback - Sanitizes user passwords (converts plaintext to BCrypt hashes) - Validates database schema columns - `getConnection()`: Returns a connection from the pool - `isDataSourceRunning()`: Checks if data source is available - `closeDataSource()`: Properly closes the connection pool

Usage: Used by all DAO classes to obtain database connections. Central point for database configuration and connection management.

`HashGenerator.java` **Location**: `src/main/java/com/crimemanagement/util/HashGenerator.java`

Purpose: Utility class for generating and testing BCrypt password hashes.

Key Functionality: - `main()` method demonstrates password hashing - Generates BCrypt hashes for passwords - Verifies password against hash

Usage: Standalone utility for generating password hashes. Can be run independently to create password hashes for database insertion.

Configuration Files

`application.properties`

Location: `src/main/resources/application.properties`

Purpose: Spring Boot application configuration file.

Key Settings: - Server port: 8080 - MySQL database connection details - JPA/Hibernate configuration (ddl-auto, show-sql, dialect) - Thymeleaf template caching - Logging levels for application and Spring components

Usage: Used by Spring Boot framework for application configuration. Note: This is primarily for web functionality; the desktop app uses `database.properties` for database configuration.

`database.properties`

Location: `src/main/resources/database.properties`

Purpose: Database connection configuration file for the desktop application.

Key Settings: - Database URL (JDBC connection string) - Database username and password - JDBC driver class name - Connection pool settings (maximum, minimum, timeouts, leak detection)

Usage: Loaded by `DatabaseConnection` class to configure HikariCP connection pool. Can be overridden by environment variables.

`logback.xml`

Location: `src/main/resources/logback.xml`

Purpose: Logback logging configuration file.

Key Configuration: - Console appender: Logs to console with timestamp and thread info - File appender: Logs to `logs/crime-management.log` with daily rollover - Log retention: 30 days, 1GB total size cap - Log levels: INFO for root, DEBUG for application packages

Usage: Configures logging behavior for the entire application. Logs are written to both console and file for debugging and auditing.

Database Files

`schema.sql`

Location: `src/main/resources/schema.sql`

Purpose: SQL script containing the complete database schema definition.

Key Components: - DROP TABLE statements for clean database setup - CREATE TABLE statements for all entities: - `officers`: Police officer records - `users`: User account records - `criminals`: Criminal records - `firs`: First Information Reports - `cases`: Case records - `suspects`:

Case-criminal linking table - `evidence`: Evidence records - `case_updates`: Case progress/update records - Foreign key constraints - Indexes for performance - Default values and constraints

Usage: Used to initialize the database schema. Automatically loaded when using H2 fallback database. Can be manually executed for MySQL setup.

`sample_data.sql`

Location: `src/main/resources/sample_data.sql`

Purpose: SQL script containing sample/test data for development and testing.

Key Components: - INSERT statements for sample officers, criminals, FIRs, cases, and users - Provides initial data for testing the application - Includes test users with different roles

Usage: Automatically loaded when using H2 fallback database. Can be manually executed for MySQL to populate test data.

`setup_database.sh`

Location: `setup_database.sh`

Purpose: Shell script for setting up the MySQL database.

Key Functionality: - Creates the database if it doesn't exist - Executes `schema.sql` to create tables - Optionally loads `sample_data.sql` for test data - Handles MySQL connection and authentication

Usage: Run manually to initialize the MySQL database: `./setup_database.sh`

`setup_database_fixed.sql`

Location: `setup_database_fixed.sql`

Purpose: Alternative/fixed version of database setup SQL script.

Key Functionality: - Contains corrected schema definitions - May include fixes for schema issues - Alternative to `schema.sql` for troubleshooting

Usage: Can be used instead of `schema.sql` if there are schema-related issues.

`reset_database.sql`

Location: `reset_database.sql`

Purpose: SQL script to reset/clean the database.

Key Functionality: - Drops all tables - Recreates the schema - Useful for resetting the database to a clean state

Usage: Run manually to reset the database: `mysql -u root -p crime_management < reset_database.sql`

`setup_password.sh`

Location: `setup_password.sh`

Purpose: Shell script for setting up database passwords.

Key Functionality: - Configures MySQL user passwords - Updates `database.properties` with password settings - Handles password encryption and security

Usage: Run manually to configure database passwords: `./setup_password.sh`

Script Files

`test_system.sh`

Location: `test_system.sh`

Purpose: Interactive shell script for testing and running the system.

Key Functionality: - Provides menu options for: - Running the application - Running tests - Database setup - System verification - Automates common tasks

Usage: Run manually for interactive system testing: `./test_system.sh`

Documentation Files

`README.md`

Location: `README.md`

Purpose: Main project documentation file.

Key Content: - Project overview and features - Technology stack - Installation and setup instructions - Database configuration - Running the application - Project structure overview - Usage guidelines

Usage: Primary documentation for developers and users. Should be read first when setting up the project.

`README_WEB.md`

Location: `README_WEB.md`

Purpose: Documentation for web-based functionality (if applicable).

Key Content: - Web interface setup - API documentation - Web-specific configuration

Usage: Documentation for web interface features (may be legacy or for future use).

`DEMO.md`

Location: `DEMO.md`

Purpose: Demonstration or example documentation.

Key Content: - Demo scenarios - Usage examples - Feature walkthroughs

Usage: Provides examples and demonstrations of system features.

`PROJECT_STRUCTURE.md`

Location: `PROJECT_STRUCTURE.md`

Purpose: Detailed documentation of project file and folder structure.

Key Content: - Complete directory tree - One-liner explanations for each directory - Organization of source code, resources, and configuration

Usage: Reference document for understanding the project layout and organization.

`UML_DIAGRAMS.md`

Location: `UML_DIAGRAMS.md`

Purpose: UML diagrams documenting system design.

Key Content: - Use Case Diagram - Class Diagram - Sequence Diagrams - Component Diagram - Activity Diagrams - State Diagrams

Usage: Visual documentation of system architecture, relationships, and workflows.

Test Files

`SmokeTest.java`

Location: `src/test/java/SmokeTest.java`

Purpose: Basic smoke test to verify system functionality.

Key Functionality: - Creates a test officer - Creates a test FIR - Creates a test case - Verifies basic CRUD operations work - Provides quick verification that the system is functioning

Usage: Run to quickly verify that database connections and basic operations are working: `mvn test` or run directly as a Java main class.

Summary

This Crime Management System is a comprehensive Java desktop application built with: - **Swing UI** for the desktop interface - **MySQL database** (with H2 fallback) for data storage - **Maven** for build and dependency management - **Role-based access control** (ADMIN, OFFICER, COURT) - **Complete CRUD operations** for all entities - **Search and filtering** capabilities - **Audit trails** and logging

The system follows a layered architecture: 1. **Entity Layer**: JPA entities representing database tables 2. **DAO Layer**: Data access objects for database operations 3. **Service Layer**: Business logic and validation 4. **UI Layer**: Swing panels for user interaction 5. **Utility Layer**: Helper classes for common operations

Each file plays a specific role in this architecture, ensuring separation of concerns, maintainability, and scalability.